

Python - Keyword Arguments

Keyword Arguments

Python allows to pass function arguments in the form of keywords which are also called named arguments. **Variables** in the **function definition** are used as keywords. When the function is called, you can explicitly mention the name and its value.

Calling Function With Keyword Arguments

The following example demonstrates keyword arguments in Python. In the second function call, we have used keyword arguments.

```
# Function definition is here
def printinfo( name, age ):
    "This prints a passed info into this function"
    print ("Name: ", name)
    print ("Age ", age)
    return

# Now you can call printinfo function
# by positional arguments
printinfo ("Naveen", 29)

# by keyword arguments
printinfo(name="miki", age = 30)
```

It will produce the following **output** –

```
Name: Naveen  
Age 29  
Name: miki  
Age 30
```

Order of Keyword Arguments

By default, the function assigns the values to arguments in the order of appearance. However, while using keyword arguments, it is not necessary to follow the order of formal arguments in function definition. Use of keyword arguments is optional. You can use mixed calling. You can pass values to some arguments without keywords, and for others with keyword.

Example

Let us try to understand with the help of following function definition –



```
def division(num, den):  
    quotient = num/den  
    print ("num:{} den:{} quotient:{}".format(num, den, quotient))  
  
division(10,5)  
division(5,10)
```

Since the values are assigned as per the position, the output is as follows –

```
num:10 den:5 quotient:2.0  
num:5 den:10 quotient:0.5
```

Example

Instead of passing the values with positional arguments, let us call the function with keyword arguments –



```
def division(num, den):
    quotient = num/den
    print ("num:{} den:{} quotient:{}".format(num, den, quotient))

division(num=10, den=5)
division(den=5, num=10)
```

Unlike positional arguments, the order of keyword arguments does not matter. Hence, it will produce the following output –

```
num:10 den:5 quotient:2.0
num:10 den:5 quotient:2.0
```

However, the positional arguments must be before the keyword arguments while using mixed calling.

Example

Try to call the division() function with the keyword arguments as well as positional arguments.



```
def division(num, den):
    quotient = num/den
    print ("num:{} den:{} quotient:{}".format(num, den, quotient))

division(num = 5, 10)
```

As the Positional argument cannot appear after keyword arguments, Python raises the following error message –

```
division(num=5, 10)
                ^
```

SyntaxError: non-keyword arg after keyword arg

TOP TUTORIALS

Python Tutorial
Java Tutorial
C++ Tutorial
C Programming Tutorial
C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial



Chapters ▾

Categories ≡

Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification

Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.